

```

// Pure, client-side proximity logic. Takes
// the user's current position and
// a list of cached camera nodes, and
// returns any nodes within the alert
// threshold. Nothing in this module
// performs network I/O or persists
// location – it is a stateless calculation
// over data already in memory.

import { point } from '@turf/helpers';
import distance from '@turf/distance';
import { PROXIMITY_THRESHOLD_METERS } from
'../utils/constants';

// nodes: [{ osmId, lat, lon,
// surveillanceType, direction }]
// userLocation: { latitude, longitude }
// Returns nodes within
PROXIMITY_THRESHOLD_METERS, each annotated
with
// its computed distance in meters, sorted
nearest-first.
export function
findNodesWithinThreshold(userLocation,
nodes, thresholdMeters =
PROXIMITY_THRESHOLD_METERS) {
  if (!userLocation || !nodes?.length)
return [];

  const userPoint =

```

```

point([userLocation.longitude,
userLocation.latitude]));

    const withDistance = nodes.map((node) =>
{
    const nodePoint = point([node.lon,
node.lat]);
    const distMeters = distance(userPoint,
nodePoint, { units: 'meters' });
    return { ...node, distanceMeters:
distMeters };
});

    return withDistance
        .filter((n) => n.distanceMeters <=
thresholdMeters)
        .sort((a, b) => a.distanceMeters -
b.distanceMeters);
}

// Tracks which node IDs have already
triggered an alert this "approach",
// so we fire once per node per approach
rather than every check-interval
// tick while the user is stationary near
it.
export function createAlertDeduper() {
    const alreadyAlerted = new Set();

    return {
        // Returns only the subset of
nodesInRange that haven't alerted yet,
        // and marks them as alerted.
        getNewlyTriggered(nodesInRange) {

```

```
        const fresh = nodesInRange.filter((n)
=> !alreadyAlerted.has(n.osmId));
        fresh.forEach((n) =>
alreadyAlerted.add(n.osmId));

        // Clear entries for nodes no longer
in range so they can re-trigger
        // if the user leaves and comes back.
        const currentIds = new
Set(nodesInRange.map((n) => n.osmId));
        for (const id of alreadyAlerted) {
            if (!currentIds.has(id))
alreadyAlerted.delete(id);
        }

        return fresh;
    },
};
}
```